

Plano e Execução de Testes de Software

Projeto: EscolaAlerta

Dupla: Mateus Farias Bentes e Mariana Idalina Bezerra Vidinho

1. Introdução

Este documento define a estratégia de testes do EscolaAlerta nos três níveis solicitados: unitário, integração e sistema. Com os casos de teste planejados, os requisitos cobertos por cada um e uma coluna de status para registro da execução. O objetivo é garantir que o motor de correlação, a API e o módulo de intervenção orientada sejam validados de forma incremental, do componente isolado ao fluxo completo do usuário.

2. Estratégia de Testes

A estratégia segue a pirâmide de testes: a maior parte dos casos é unitária (rápida e barata de manter), um conjunto menor cobre a integração entre camadas da API, e poucos casos de sistema validam o fluxo completo: app Flutter, API e banco de dados funcionando juntos. Essa proporção reduz o tempo de execução da suíte sem abrir mão da cobertura sobre o comportamento observável pelo usuário.

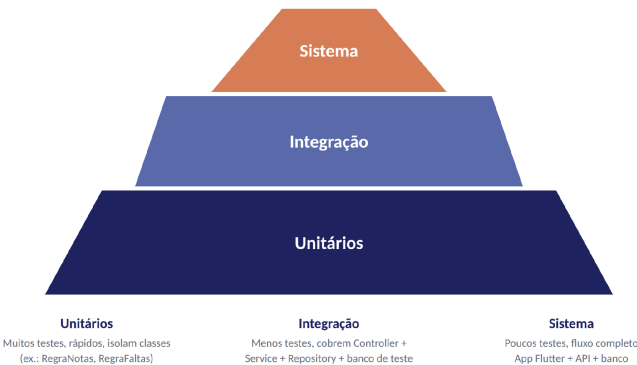


Figura 1: Pirâmide de testes aplicada ao EscolaAlerta.

Cada caso de teste abaixo já nasce com status "Pendente"; atualize para "Aprovado" ou "Reprovado" (com uma nota sobre a falha) conforme os testes forem executados. Assim o mesmo documento serve como plano e como registro da execução.

3. Testes Unitários

Escopo: classes e funções isoladas, sem acesso a banco de dados ou rede, principalmente as estratégias do motor de regras (Strategy) e funções de validação.

Ferramentas: Jest para a API (Node.js); flutter_test para os widgets e funções do app Flutter.

ID	Unidade testada	Cenário / resultado esperado	Requisito	Status
UT01	RegraNotas.avaliar()	Aluno com média abaixo do limite em 2 disciplinas consecutivas deve ser sinalizado como risco.	RF07	Pendente
UT02	RegraFaltas.avaliar()	Aluno com faltas acima do limite no período deve ser sinalizado como risco.	RF05 / RF07	Pendente
UT03	RegraOcorrencias.avaliar()	Aluno com 3 ou mais ocorrências no mês deve ser sinalizado como risco.	RF04 / RF07	Pendente

ID	Unidade testada	Cenário / resultado esperado	Requisito	Status
UT04	MotorDeRegras.classificar()	Combinação de múltiplos sinais de risco deve retornar o nível correto (baixo, moderado ou alto).	RF07	Pendente
UT05	MotorDeRegras.classificar()	Nenhum sinal de risco presente deve retornar nível "baixo".	RF07	Pendente
UT06	Validação do cadastro de aluno	Requisição sem CPF ou dado obrigatório deve retornar erro de validação (400), sem chamar o Repository.	RF02	Pendente
UT07	Hash de senha (bcrypt)	A senha em texto puro nunca deve ser persistida; apenas o hash deve ser gerado e comparado.	RNF01	Pendente
UT08	GeradorDePlanoDeAcao	A partir de uma causa raiz de risco (ex.: faltas), deve retornar a sugestão pedagógica correspondente.	RF12	Pendente

4. Testes de Integração

Escopo: comunicação entre Controllers, Services e Repositories, e destes com um banco de dados de teste (PostgreSQL isolado ou em contêiner), sem envolver o app Flutter.

Ferramentas: Jest + Supertest, executando requisições HTTP reais contra a API em ambiente de teste.

ID	Endpoint / fluxo testado	Cenário / resultado esperado	Requisito	Status
IT01	POST /alunos	Cria um aluno vinculado a um responsável autenticado e persiste corretamente no banco de teste.	RF02	Pendente
IT02	POST /notas + GET /alunos/:id/notas	Nota registrada é recuperada com os mesmos valores de disciplina, período e nota.	RF03	Pendente
IT03	POST /login (credenciais válidas)	Retorna token JWT válido para um responsável cadastrado.	RF01 / RNF01	Pendente
IT04	POST /login (credenciais inválidas)	Retorna erro 401 e nenhum token é emitido.	RF01 / RNF01	Pendente
IT05	Registro de faltas recorrentes	Ao ultrapassar o limite configurado, o motor de regras dispara um alerta persistido no banco.	RF05 / RF07 / RF08	Pendente
IT06	GET /alertas/:alunoId	Retorna o histórico de evolução de risco armazenado, em ordem cronológica.	RF08	Pendente
IT07	POST /alertas/:id/feedback	Registra o feedback de resolução do responsável e realimenta o histórico do aluno.	RF13	Pendente

5. Testes de Sistema

Escopo: fluxos completos do ponto de vista do usuário, com o app Flutter, a API e o banco de dados operando em conjunto.

Abordagem: execução roteirizada (manual) a partir dos cenários abaixo, com possibilidade de automatizar os principais fluxos futuramente com o pacote `integration_test` do Flutter.

ID	Cenário	Resultado esperado	Requisito	Status
ST01	Login, cadastro do filho e painel inicial	Responsável faz login, cadastra o aluno e visualiza o painel sem alertas pendentes.	RF01 / RF02 / RF08	Pendente
ST02	Alerta automático por faltas	Escola registra faltas consecutivas; responsável recebe alerta no app com o nível de risco correspondente.	RF05 / RF07 / RF08 / RNF06	Pendente
ST03	Plano de ação e feedback	Alerta gerado exibe o plano de ação sugerido; responsável registra feedback de resolução com sucesso.	RF12 / RF13 / RNF07	Pendente
ST04	Controle de acesso entre contas	Responsável autenticado não consegue visualizar dados de aluno vinculado a outra conta.	RNF01 / RNF02	Pendente
ST05	Uso em mobile e web	As principais telas do EscolaAlerta funcionam corretamente tanto na versão mobile quanto na web (mesma base Flutter).	RNF05	Pendente

6. Rastreabilidade Requisito × Teste

A tabela a seguir relaciona cada requisito do EscolaAlerta aos casos de teste que o cobrem, permitindo verificar a cobertura antes de cada entrega. Requisitos ainda sem teste associado devem ser detalhados na próxima rodada de planejamento.

Requisito	Descrição	Casos de teste
RF01	Autenticar responsáveis via login seguro	IT03, IT04, ST01
RF02	Cadastrar alunos vinculados aos responsáveis	UT06, IT01, ST01
RF03	Registrar notas por disciplina e período	IT02
RF04	Registrar chamadas de atenção (ocorrências)	UT03
RF05	Registrar frequência e faltas	UT02, IT05, ST02
RF06	Registrar saídas antecipadas	(a detalhar na próxima rodada de testes)
RF07	Aplicar motor de regras e classificar o risco em baixo, moderado ou alto	UT01, UT02, UT03, UT04, UT05, IT05, ST02
RF08	Gerar alertas automáticos e exibir painel com histórico de evolução	IT05, IT06, ST01, ST02
RF12	Gerar plano de ação a partir da causa raiz do alerta	UT08, ST03
RF13	Registrar feedback de resolução e realimentar o histórico	IT07, ST03
RNF01	Segurança: autenticação JWT e senha criptografada (bcrypt)	UT07, IT03, IT04, ST04
RNF02	Conformidade com a LGPD: dados fictícios e controle de acesso	ST04
RNF03	Manutenibilidade: código modular, versionado em Git/GitHub	Verificação de processo (revisão de código), não coberta pelos testes automatizados
RNF04	Precisão: motor de regras avaliado frente a cenários simulados (meta: 80% de acerto)	UT01–UT05 (conjunto de cenários simulados)
RNF05	Disponibilidade: aplicação web e mobile, via Flutter	ST05
RNF06	Desempenho: API REST com bom tempo de resposta	ST02

Requisito	Descrição	Casos de teste
RNF07	Explicabilidade: alerta e sugestão rastreáveis à regra que os originou	ST03

7. Plano de Execução

Os testes unitários são executados a cada alteração de código, antes de qualquer integração ao repositório principal, idealmente de forma automatizada a cada commit. Os testes de integração são executados ao final de cada sprint, antes de mover os itens correspondentes para "Testado" no quadro Kanban do projeto. Os testes de sistema são executados antes de cada entrega para a professora e, de forma completa, antes da apresentação final à banca.

8. Considerações Finais

Este plano cobre os requisitos funcionais e não funcionais já levantados no pré-projeto, com ênfase no motor de correlação e no módulo de nudge, que concentram o maior risco técnico do EscolaAlerta. A cada nova funcionalidade implementada, novos casos devem ser adicionados nos três níveis, mantendo a tabela de rastreabilidade atualizada.

Referências

INTERNATIONAL SOFTWARE TESTING QUALIFICATIONS BOARD (ISTQB). Certified Tester Foundation Level Syllabus. Version 4.0. Berlim: ISTQB, 2023.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: Uma Abordagem Profissional. 8. ed. Porto Alegre: AMGH, 2016.